OCP GLOBAL SUMMIT 2026 • OPEN PLATFORM FIRMWARE

在固件栈上扩展内存安全

> OCP 圆桌：Oracle · Microsoft · Insyde · Google · AMI · AMD

时长 32 分 50 秒

来源 youtube.com/watch?v=hVk3Z95_QEY

形式 圆桌讨论，无幻灯片

在固件栈上扩展内存安全：OCP 圆桌实录

> OCP Global Summit 2026 · Open Platform Firmware (OPF) 圆桌 主持：Sam Mutu（Oracle 核心固件工程架构师） 嘉宾：Giri Mudusuru（Microsoft，Azure 固件）、Tim Lewis（Insyde，CTO / 安全与信任办公室负责人）、Bertrand Achard（Google，服务器软件团队负责人）、Oleg Ilyasov（AMI，BIOS 工程师）、Raj Kapoor（AMD，OPF 项目负责人） 全长 32 分 50 秒 ｜ **本场为圆桌讨论，全程无幻灯片**，故以"议题分段 + 中文译文 + 英文原句 + 小结"的三层结构呈现

背景　安全启动解决了什么，没解决什么

主持人开宗明义：固件认证与安全启动解决的是"固件从哪来"，而不是"固件内部是否安全"。这一区分是整场讨论的出发点。

中文译文　今天我们要谈的是在固件栈上扩展内存安全。先说个起点：我们确实有固件认证、安全启动、认证启动这些东西——它们真正保证的是，你交给系统的固件来自一个经过认证的版本，你不想运行未经验证的固件。但它并不提供固件内部本身的安全性。尽管我们已经把固件部署了好几代，但因为新语言、新机制不断出现，我们需要重新审视，看看在固件领域、在整个栈上能做什么——不只是我们，可能还有 BMC，以及任何我们写固件的地方，它可以是任何微控制器，对吧？所以我们觉得可以组织一场圆桌，讨论从这里能往哪走。

> "today we're going to talk about scaling memory safety across firmware stack. um to kind of start with right we do have a uh firmware authentication secure boot authenticated boot those are really take make sure that the firmware that you're giving to the system it's uh coming from authenticated uh version you don't want to run authenticated firmware but it does not provide the security inside the firmware itself um even though we have deployed the firmware for multiple generations s uh because of new languages and new mechanisms are showing up. We need to kind of revisit and see what we can do in the firmware area across stack not just by us probably even BMC and wherever the we are writing firmware it can be microcontroller anywhere right so we thought we can have a panel and can discuss about what we can do go from here right"

> **小结**　安全启动只保证"来源可信"，不保证"实现无缺陷"。随着新语言与新机制出现，固件层需要重新审视——且范围不止于主机固件，还包括 BMC 与各类微控制器。

嘉宾

主持人依次介绍：**Sam Mutu**（Oracle，核心固件工程架构师，视野覆盖从网格到机架，以及机架内部分固件）；**Giri Mudusuru**（Microsoft，Azure 固件，端到端负责从存量维持到新产品）；**Tim Lewis**（Insyde CTO，从业 30 年，兼安全与信任办公室负责人，负责处理所有进来的安全事件报告）；**Bertrand Achard**（Google，领导"服务器软件"团队，负责 BIOS 等固件组件的部署与维持）；**Oleg Ilyasov**（AMI，BIOS 工程师，涉及启动固件的各方面与相关模块开发）；**Raj Kapoor**（AMD，与 Sam 共同领导 OPF 项目，主职是长期战略举措）。

> **小结**　六位嘉宾覆盖了固件生态的完整链条：芯片（AMD）、BIOS 厂商（AMI、Insyde）、超大规模用户（Microsoft、Google）、系统厂商（Oracle）。这一组合使后续讨论不停留在"谁该负责"，而是能落到各自可执行的动作。

一　为什么"现在"要关注固件内存安全

1.1　增量改良已经推不动了

中文译文　就在我走上这个讲台之前，我在想：我们时不时会走到一个点，那时增量式的改动已经推不动指针了，于是我们必须换一种完全不同的方式去思考。固件大概就处在这样一个位置。这也是为什么我们在琢磨那个老掉牙的问题——内存安全。那么，我们怎么解决它？我们必须想得不一样一点。怎么做到？

> "every um now and then we get to a point where the incremental changes just don't move the needle and so we have to start thinking about things very differently. So firmware is kind of such place that it is in and um so we which is why we are pondering over the age-old question if you will right memory safety. So how do we how do we solve that? So we have to think a bit uh differently and how do we do that?"

> **小结**　内存安全不是一个新问题，而是"老问题 + 增量手段失效"。Raj 的论断是：继续做局部改良不会改变结果，必须换方法。

1.2　"完美风暴"：三个因素同时到来

中文译文　我们目前在这个问题上正处于一场"完美风暴"之中。因为就在固件相关问题的数量出现尖峰的同时——这些是 AI 发现的。如果你现在还没关注 Mythos，你应该去关注，因为我们通常一年会遇到两波尖峰，那些通常来自我们芯片厂商的朋友们。但最近三个月，我们遇到了第三波尖峰，而这一波是 AI 发现的安全漏洞，其规模与我们传统上从其他地方看到的相当。

同时，这些问题中有 60% 到 70% 是内存问题——Google 的研究表明 60% 到 70% 的安全问题与内存相关。第三件事是，我们有来自欧盟等地的监管要求，它们在说："顺便说一句，如果你有安全问题，你的时间非常有限，你必须披露，而且你修复它的时间非常有限。"所以这些事情恰好在同一时刻汇合到一起。它基本上是在说：如果我们不解决其中一个问题，我们余生就都在做安全缺陷修复了。

而且不只是欧盟，我想美国也一样——他们提出来了，后来撤销了一些，但没错，世界各地的监管要求正在把安全变成产品出货的根本条件，这就是这里"完美风暴"的第三部分。

> "We we're currently kind of a perfect storm for this issue as well because at the same time that we have the spike in issues uh in firmware related to um AI discovered. Um if you're not looking at Mythos right now, you should be because we usually get two spikes a year and those come usually from our silicon vendors friends. But now we fin this last three months we've have a third spike and that spike is uh AI discovered security vulnerabilities equal in size to the ones we've traditionally had from other places. At the same time we have 60 and 70% of those issues are memory issues as Google has done studies that show that 60 to 70% of uh security issues are memory related. And the third thing is we've got regulatory requirements from places like the EU which are saying, 'Hey, by the way, if you have a security issue,

> you've got very limited, you must disclose it and you have very limited time to fix it.' So those things are coming together at the exact same time. And it's basically saying if we got if we don't solve one of these problems, we're going to be we'll be doing security bug fixes for the rest of our lives. >> And it's not just EU, I think it's uh US as well. uh right they came up with it and then they resended some but yes they're >> regulatory requirements from around the world are driving security as a fundamental thing shipping your product and that's that's the part of the perfect storm here of three things"

> **小结** 三重叠加：①AI 发现的漏洞形成"第三波尖峰"（历史上每年两波，均来自芯片厂商）；②其中 60–70% 属内存类问题；③欧盟等地监管要求强制披露与限期修复。结论很直白——不消除这一类缺陷，团队将永远在做安全补丁。

1.3 超大规模运营者的视角：存量才是主战场

中文译文 从整个行业来看——是的，正如 Raj 和 Tim 总结的，我们看到的是一场完美风暴。但这不是什么新问题：编译器安全、编译期构建时的静态检测等等，这些都已经有了。但正如 Raj 强调的，这不是增量式地看，而是退后一步看，看怎么解决它。因为对我们超大规模运营者来说，其中大部分是存量舰队的维持，而不只是未来——因为 Tim 刚强调的所有这些安全修复，都是我们必须去修的挑战；再加上 FedRAMP 要求和关键的时间 SLA，这对我们变得极其关键，而且从人力上是不可能扩展的。

> "from the whole industry right yes as you summarized Raja and Tim right uh it's a perfect storm as we are seeing but this is not the new thing to address your question is right it has been happening like compiler safety like compiler level buildtime static detections and all are there but this is like looking at and revisiting like Raj just highlighted is it's not just incremental looking at taking a step back and looking at it how to solve it because majority of this as a hyperscaler uh is the sustaining fleet not just the future because all these security fixes uh Tim just highlighted are the challenges that we have to go fix and with fed ramp requirements and critical timing SLAs and this becomes a very critical part of us and it is not humanly possible to scale."

> **小结** 超大规模运营者的成本结构决定了优先级：不是"未来新平台"，而是"已部署舰队的维持"。在 FedRAMP 要求与关键 SLA 时限下，人工逐个修的方式在规模上不可行。

1.4 为什么固件层比操作系统层更危险

中文译文（Oleg Ilyasov，AMI） 这些安全漏洞有不同种类。如果它们在操作系统层被发现——系统正在运行、发现那里有个漏洞——那只是更新操作系统、希望一切正常的问题。然而，我们越往栈的下面走，如果到了启动固件，在那里发现安全漏洞，那就危险得多，而且要把修复部署到各个系统、各个平台、各个厂商，也困难得多。所以我们真的需要尽可能早地去对抗它，尽可能多地去思考它。

中文译文（Bertrand Achard，Google） 如果我们回到今天的演讲，我们知道我们在谈内存安全——如你所说，它是固件中安全漏洞的一个非常主要的成因。在座很多人对此熟悉，但实际上很多人并不熟悉，他们倾向于忘掉它、忘掉它的存在。它就像在后台，有时只在最开始运行一下，而这使它特别危险：不仅很多人忘了它，而且它极其强大——一个固件漏洞就能彻底攻陷一个系统。所以业界对内存安全、对固件整体安全保持一定的意识是非常好的，因为这样我们才能让整个平台变得更好。

> "the security vulnerabilities are different uh kinds uh if if there if they found in the OS level if the system is running and it is found that there is a vulnerability there it's only a matter of updating the OS and hoping that everything is fine whereas so the lower down the stack we go if we go to the boot firmware if the security vulnerability is found there it's a lot more dangerous and it's a lot harder to deploy the fix across the systems across different platforms across different vendors etc. So we really need to fight it as early as we can. We really need to think about it as much as we can."
> "if if we go back to the talk today we know we talk about memory safety like you mentioned it's a major major cause of security vulnerabilities and firmware and firmware many people here are familiar with it but actually a lot of people are not they tend to forget it and forget it exists. It's like in the background it runs only at the beginning sometimes and that makes it particularly dangerous because not only a lot of people forget about it but also it's extremely powerful. you can absolutely completely compromise a system with a firmware vulnerability. So that's why it's very good to have some awareness in the industry about memory safety and general security across the firmware because that's how we're going to make the whole platform better."

> **小结**　两个叠加因素：①越往栈底，修复越难——OS 补丁只需更新，固件缺陷要跨平台、跨厂商部署；②固件"只在启动时运行、平时不可见"，因此被系统性忽视，但其权限足以完全攻陷系统。这正是"被忽视的高权限层"。

二　只换语言重编译就够了吗

主持人提出关键追问：是不是选对语言、重编译一次，安全问题就解决了？还是需要做更多？如果只能选一门语言，哪门最好？

2.1　真正的判据是"何时发现"，不是"用哪种语言"

中文译文（Oleg Ilyasov，AMI）　这不是"有没有最好的语言"的问题，而更在于：我们怎么以一种方式解决它，使我们在代码编译之后、在单元已经出货到现场之后、然后它跑出问题之后，才发现它？那时我们再去调试，调试变得非常棘手，过了很多很多个月，我们才意识到固件里存在那个漏洞。

（主持） 有时是很多年。 **（Oleg）** 有时甚至好几年——没错。它可以一直不被注意到。所以我们需要一种方法，能在编译期就检测出这类内存安全问题——那会是最好的情况。而且外面确实有现成方案，但准确回答这个问题：关键不是语言，而是我们想在哪一环产生影响。我们需要的方案是，赶在它浮现之前。

（Oleg 续） 一个办法就是评审、代码同行评审；还有一些启发式的源码分析器可以跑，找出可能存在的问题。但即便修好了，如你所说，也不能保证一年之后另一个模块被开发出来时，不会一次又一次地引入类似问题。而正如 Raj 所说，如果语言本身原生支持这类问题的"根治"，那对我们来说是非常非常有吸引力的路径。所以关键在编译期——这意味着你需要语言层面的支持。

（Tim Lewis，Insyde） 你编译不过它——如果有问题，它就编译不过，就是这样。这正是我们想要的。

> "It's not a question of you know having the best language but it's more of the question of how do we how do we solve it in a way that we don't find it after the code has been compiled after the the the unit has been shipped into the field and it then runs into an issue. We then debug and debug becomes hairy and after many many months we realize the vulnerability that exists in the firmware. >> Sometimes many years >> sometimes even years that is correct. It can go unnoticed and u so what we need is a method to be able to detect such issues such memory safety issues uh during

> compile time. that would be the best case right and we do have solutions out there but uh precisely to answer the question it's not the language it is uh it is where we want to impact right and the solution that we need is um is is to get ahead of when it surfaces"
> "One is just a review a peer review of the code and there is some huristic uh source code analyzers that can be executed and find the potential issues that that can be there even if it's fixed as uh as you mentioned it's not a guarantee that a year after that when another model is developed there will be similar problem introduced again and again and again and like Raj mentioned the language if it supports natively the the cure for this kind of thing is very very attractive approach for us. So it's a compile time that means you need to have language support to >> you cannot compile it if it's if it's >> if it has an issue it won't compile period that's what we want"

> **小结**　判据不是"最好的语言"，而是"把发现时点前移到编译期"。人工评审与启发式静态分析都只能覆盖当次，一年后新模块可能重现同类缺陷；语言级保证的价值在于"有问题就编译不过"——这是从"检查"到"不可能写错"的跃迁。

2.2　强制力的价值：让编译器替管理者说话

中文译文（Tim Lewis）　因为没有一个好办法——如果你手下全是这些 BIOS 工程师，我自己也是一样的处境——你总不能跟所有人说：抱歉，你得停下来，这段代码不能出货。而他们说：但我需要写这段代码，它必须能工作。你说：是的，但我希望它既能工作又安全。那么，如果我有一个工具能保证我写出来的东西是安全的，那简直像金子一样。因为现在我得去跟我的工程师们较劲，而他们非常擅长自我欺骗、告诉自己"我的代码是完美的"，但我们知道那并不真。所以，如果能让编译器来说这句话，而不是我来说，也许我和他们的关系还能保住。

中文译文（Sam Mutu，主持）　我把它看成一个工具，但它不是解决方案，对吧？我们有 codeQL 之类的工具，跑一遍，问题被找出来。但 AI 类工具的一个主要问题是，开发者会说"这不适用于我们"，很轻易就写个理由绕过——因为他们可能看不到全貌。而有了强制机制就更容易，我们不需要人去逐项检查。静态分析工具、MISRA、CWE 这些东西，它们在那儿做个粗略检查，但正如我们过去这么多年看到的，它们并不十分有效。

> "because there's no good way for if you got all these engineer bias engineers because I'm in the same boat right you got all these bias engineers you're going to tell them sorry you need to stop >> and you can't ship this code and they said but I need to write this code that has to work and they said yes but I want it to work and be safe well if I can have a tool that will guarantee that when I write the it will be safe. That that would be that's like that's like gold because right now I'm I'm having to I'm having to fight my engineers and and and they they are very good at lying to themselves and telling themselves, 'Oh, my code is perfect, but we know it's not true.' So, if I can have the compiler say it instead of me having to say it, maybe my relationship with them will be saved."
> "I see this as a tool, but it's not a solution, right? We have tools like the code QQLs and all we run and the issues are found. But one of the major thing right with the uh AI based tools and all is developers makes oh it doesn't apply to us just easily write off right bypasses but because they may not see the entire thing and having this enforcement makes it easier and we don't need monitors to go check each and everything static static analysis tools and mistra and coarities you know they they are there for cursory check but uh you know they they not very effective as we all have seen in the past so many years."

> **小结**　两个层面的洞察：管理上，编译期强制把"人管人"变成"机器管规则"，消解了安全要求与交付压力之间的长期拉锯；技术上，静态分析与合规工具容易被"这次不适用"轻易绕过，只有强制机制才具备不可协商性。

三　指针与 unsafe：如何让风险可审计

主持人追问：即便做到编译期检查，固件里大量使用指针，怎么处理？

3.1　不是消灭不安全代码，而是压缩它的比例

中文译文（Tim Lewis）　显然，这正是某些语言（比如 Rust）试图解决的问题——它们试图让它……不，你不能把它变成非法。它仍然可以有不安全代码，但至少你可以把不安全代码收窄到占总量的一个越来越小的比例。随着这个比例下降，意味着攻击面下降，也就意味着总体上我得到的代码更安全。

所以有些语言会给我们这个优势，我们正在看：Rust 是那条路吗，还是有更好的办法？但我们必须选定一个，因为如果现在不开始，我们永远不会变得更好。

> "obviously that's some things like what some languages like rust are trying to solve right they're trying to make it >> elite Not you can't make it illegal. It can still be unsafe code but at least you can narrow down the unsafe code to a smaller and smaller segment of the total percentage and as that percentage goes down it means the attack surface goes down and that means that I'm in general I'm getting safer code."
> "So some languages will give us that advantage and we're so we're looking at how do you is Rust the way that does that or is there some better way but we have to choose something because we can't if we don't start now we will never get any better."

> **小结**　Rust 的价值不在于"消灭不安全"，而在于把不安全部分变成可度量的小比例，从而让攻击面随比例下降而收缩。这也解释了为什么"必须现在就选定一条路"——路线可以再调整，但没有起点就永远没有改善。

3.2　显式标注使风险可检索、可追责

中文译文（Tim Lewis）　关于某些语言（比如 Rust），有一点是：你必须显式标注它。你不标，它就不工作。所以这是一个很大的红旗。如果我要做代码评审，第一件事就是去看每一个 unsafe 段，然后问：为什么这里不安全？他们必须向我解释为什么把它留成那样。如果他们不解释——

（主持） 而且作为开发者，你得亲手敲下 `unsafe` 这个词。 **（Tim）** 是的，你为此负责。你必须理解为什么。为什么——你为什么把前门的锁留着没锁？你必须知道为什么。

中文译文（Sam Mutu）　但从编译器这一侧看，编译器里也有一些强制检查。可是涉及结构布局和更大的编译单元时，编译器的标志会在某处被覆盖掉。所以这不是一件容易审计的事。而显式标注使它更容易被审计——然后那可以成为我们逐条追踪的东西。初期阶段也许就是先达到同样的水平，然后再做监控。所以你大概可以去搜一遍 `unsafe`、把它弄清楚，建立一个度量并设置触发告警。

> "one thing about some languages like Rust you have to mark it explicitly right >> right you can't you can't it will not work unless you mark it explicitly and so that's a big red flag and if I'm going to do a code review first thing I'm going to look for is look at every single unsafe section and go why is it unsafe and they have to then they have to justify to me why they left it that way. They don't. >> And if you're a developer, you have to type unsafe. >> Yes. >> You're responsible for that. You have to understand why. Why is it is it that >> why did you leave the door on your front door unlocked? >> You have to know why."
> "But from the compiler side, right? There are some enforcement even in the compiler. But with the structure layouts and larger compos like the compiler flags are overridden somewhere. So it's not something easy to audit >> and this explicitly marking makes it easier auditable >> and then that could be a one we can track one by one right for initial phase it could be yeah just get this same level then monitoring >> so you can probably go and do a search of unsafe and figure out again >> create a metric and trigger."

> **小结** Rust 的关键设计不是"消除不安全"，而是"强制标注不安全"：`unsafe` 必须显式写出，于是可被检索、计数、设阈值告警，并在代码评审中逐条要求理由。这把一个不可审计的属性（"这段代码安不安全"）转成了可度量的工程指标（"unsafe 占比与清单"）。

四 迁移路径：从新代码开始，逐层推进

4.1 先新代码，再逐层攻

中文译文（Raj Kapoor，AMD） 我相信每一家想为自己解决这个问题的公司，都在琢磨攻击这个问题的最佳路线是什么。现在我们所有的固件——至少主机固件——都是 C 写的，而且代码库很大，不是小代码库。一个方案是把它转成 Rust。这可行吗？值得怀疑。但你必须从某处开始。

那么从哪里开始？在我看来，从新代码开始：你从新代码开始，并且坚决要求用 Rust 写。那么，只做新代码就能保证你没事吗？不能。但至少球滚起来了。然后你开始看你的层次：下一层技术是什么？你一层一层地攻。

当然，有 AI 工具可以帮忙——我不是说它能完整解决这个问题，但它能帮上忙。AI 工具没法去分析你的代码、发现这里有个循环图所以必须改成向量之类的。我想说的是：仍然需要人；需要做研究让 AI 变聪明；需要开发 agent，非常有条理地从一层到另一层、一层叠一层地推进。你取一层，就可以按函数为单位开始做。有很多做法，但到目前为止我还没遇到单一的解法。所以你必须把它拆碎了一块块做。

我们正处在一个拐点。人们在研究，政府也在那一侧提供帮助——DARPA 启动了一个叫 TRACTOR 的计划。它代表什么？把 Rust 翻译成 C……抱歉，是把所有 C 翻译成 Rust。这是一个资助研究的计划，所以有大学在研究，也有很多其他特别兴趣小组在做。所以业界在动。有路可走，但你必须开始。每家公司都必须开始，我相当确定大家都在看这件事——Microsoft、Google、Meta，你能想到的都算。

> "every company who is looking to to solve this problem for themselves um they they are pondering about what is the best route to attack this problem. Um now most all of our firmwares are in C. Um of at least the host firmwares. Um it is all in C right uh and it is a large code base. It's not a small code base. Now one solution would be convert it into Rust. Right? Now is that practical? Questionable. Right? So you have to but but you have to start somewhere right."

> "So where do you start in my opinion would be to start with new code. you start with new code and invariably commit yourself to write it in Rust. Right? Now, will that guarantee that um uh you'll you'll be fine with just new code? No. So, once you've you've gotten that, but at least you got the ball rolling, right? Now, you start to look at your layers, right? So, what is the next layer of tech, right? So, you start attacking one at a time."
> "there are AI tools uh that are that can assist uh I'm not saying it can u it can solve this problem in its entirety, but it can help. Right? AI tools don't have a way to um to analyze your code and figure out there's a circular graph and so it must change it to vector and so on so forth. So what I'm trying to say is that there is human that will be required. There are um uh there is research that will need to uh be done to make AI smart. There will be the need to develop uh agents if you will um to very methodically you know uh start working on layers to layers upon layers."
> "We are at an inflection point. People are researching there is government uh that is helping in that front as well. is um DARPA has initiated a program called tractor. Um what does it stand for? Um translate rust to C. >> Translate all C to sorry Rust to C. So translate all C to Rust. So it's a it's a program that funds you for research. So you you have universities looking into it."

> **小结** 路径是"新代码强制 + 逐层推进 + 按函数拆碎"，并且承认不存在单一解法。外部条件正在改善：DARPA 的 TRACTOR 计划资助 C→Rust 的转换研究，多家超大规模厂商都在投入。关键判断是"必须先开始"——路线可以迭代，起点不能缺席。

4.2 模块化：优先内存密集与高频变更的部分

中文译文（Giri Mudusuru，Microsoft） 我只是想补充：并不是说我们要把一切停下来、把所有东西重写一遍。正如 Raj 提到的，这更像是一个渐进过渡——新代码强制执行。但问题是，我们怎么把固件里那些关键部分模块化？比如那些可能频繁变更、或者不怎么变更的部分。正如我们昨天讨论的，这可能取决于具体情况，是主观的，取决于怎么开始转换。而今天你去看，即使以 EDK2 为例，里面还有很多地方仍然存在汇编。并不是说 C 会消失，但关键部分——内存密集的部分、内存管理的部分——是最需要优先处理的。

中文译文（Tim Lewis） 这是一个巨大的代码库。有一点我们必须都同意：我们会长久地待在这里。这不是一次快速转换，会持续很长时间。

> "it's not like we'll take everything stop everything and rewrite everything right I think as Raj touched upon it's more of the gradual transition the new code is make it mandatory >> but how do we modularize the critical pieces of firmware like which maybe frequently change or not change and as we were discussing yesterday uh that could be subjective based on how it is uh start converting right and today if you see that there are a lot of pieces even in EDK2 as an example where assembly still exists. It's not like C is not going to disappear but the critical portions the memory intensive portions memory management portions are the first ones that need to be prioritized."
> "It's a huge code base right one one thing we have to all agree upon is that >> we are going to be stuck here for the long haul. It's not a it's not a you know quick conversion. will be there for for a long time."

> **小结** 优先级的判据是"内存密集 + 关键路径"，而非"新旧"。EDK2 里至今仍有汇编这一事实说明：多语言共存是长期状态，因此模块边界的设计（哪些块可以独立转换）比转换工具本身更决定推进速度。

五　供应链、人才与开源：为什么这不是"某个团队的问题"

5.1　这是整条供应链的问题

中文译文（Bertrand Achard，Google）　我认为这是一个供应链问题。我说的供应链是指整条供应链：不只是软件供应链——如我们提到的，那里可能有二进制 blob（那是最糟的），或者至少有一些 C 库，你把它集成进来，就像瑞士奶酪一样，我们在内存安全的代码里嵌进了不安全的代码；还包括人才——没有那么多会写 Rust 的人，甚至有些人不相信它有用，等等。所以我们必须建设这条供应链，而这要从所有利益相关方开始：提供库的公司必须开始用 Rust 提供，而不只是 C；学编程的人——也许有了 AI 现在这事没那么要紧——但与其让 agent 写代码，不如让它用 Rust 写，也许你会惊讶。

所以我认为，这是我们作为一个行业必须解决的问题：整条供应链——人才、软件库、一切——都必须修好，而每个人都有角色要扮演。正如 Raj 所说，这不只是别人的问题。

> "I think it's a supply chain problem and when I say supply chain what I mean by that is the whole supply chain so it's not just software supply chain where you could have as we mentioned some binary blobs that's the worst or at least some C libraries that you can integrate it create like a Swiss chip we integrate on safe code with memory safe code but also talent people there's not that many people that know how to write code in Rust and and there's even some people that are not even convinced that it's useful etc so we have to build that supply chain and it starts with all the stakeholders companies that provide libraries they have to start providing them in rest not just in C people that learn coding or maybe with AI it's a bit less of a thing now but instead of asking the agent to write code ask it to write it in R maybe you'll be surprised."
> "I think that's the that's the problem we have to solve as an industry like the whole supply chain talent software libraries everything has to be fixed and everybody has a role to play like Raj mentioned it's not just someone else's problem."

> **小结**　供应链有四段：二进制 blob（最难）→ C 库 → 人才储备 → 认知共识。任何一段缺失都会让"内存安全"在上层被稀释回原样——这正是"瑞士奶酪"比喻的含义。

5.2　用开源加速采纳

中文译文（Giri Mudusuru）　而且现在的时机是对的，因为 AI 已经能完成大部分转换工作。所以更多是找到合适的那个、排出优先级。从基础设施角度，至少 Microsoft 已经做了 Patina 这部分，它是开源的，所有人都可以采用，已经就绪，我们甚至看到一些生产级别的采纳。从数据中心这边，我们正在内部评估，但关键是势头——我们怎么开始用它。也许还有别的途径，但当初做这件事的关键原因就是加速采纳，而不是把它留在内部。

中文译文（Raj Kapoor）　我是说，谈谈你们做过的事和取得的成功——你们应该开放出来。这样整个行业才能从中受益。而且样例是现成的。人们需要被教育。这是下一步。

> "the transition now is timing is correct in terms of like now a can do the majority of the conversions. So it's more of like finding the right one prioritizing is the one and from infrastructure wise at least from Microsoft we already did the patina the rest and it's open source and everybody to adopt it's ready and we see some adoptions even at a production level. So um from the data center side we are evaluating internally but it's the momentum right so how do we go and

> start using that maybe there are different other means but the key reason why that was done is to accelerate the adoption rather than keeping internal"
> "I mean you know talking about what you've done and the successes that uh you have achieved um you should be open about it. >> Yeah. >> Right. So that the industry can benefit out of it >> and samples are available. People need to be educated."

> **小结** Microsoft 把内部 Rust 组件（Patina）开源，理由是"加速采纳而非内部闭环"。这一选择隐含一个判断：在存量巨大的固件生态里，单家公司的内部实践无法形成规模效应，只有开放才能让供应链上游（库、工具、人才）跟上。

六 存量舰队、规范约束与攻击面

6.1 泰坦尼克号转不了身

中文译文（Tim Lewis） 既然我在这行干、这是我的主业，有一个问题：你没法在半小时内让泰坦尼克号掉头——在你发现冰山之前。因为如你所说，代码库非常庞大，生态里有海量已部署系统。等我们把这件事再往前推一点，还会有其他变化。比如，即便在我们开源的那些 Rust 代码里，你仍然能看到一些标记为 `unsafe` 的段落。为什么？因为它们所依据的规范——UEFI 规范——只能帮到一定程度，因为它们并不是为 Rust 这样的内存安全语言设计的。所以未来还有规范层面的工作可以做，来进一步改善这里的情况。

> "you don't turn you don't turn the Titanic in you know half an hour in half hour right. Yeah. before you spot the iceber because as you said the code base is very large and there are a huge number of deployed systems in the in the ecosystem once we get this a little bit further along with there's some other changes that could happen for example um even within the rust code that we are that that you know in the open source you can see there are still some sections marked unsafe why because the specifications that they're based on the UFI specifications only support can only help so much because they're not designed for a memory safe language like Rust. So you know there's future spec work that can be done to even improve improve things here."

> **小结** 一个容易被忽略的约束：即使全部改用 Rust，只要底层接口规范（如 UEFI）的抽象方式本身依赖指针语义，`unsafe` 段就无法消除。因此规范层也需要为内存安全语言重新设计——这是标准组织（含 OCP/OPF）可以介入的位置。

6.2 超大规模运营者的核心矛盾：要修存量，也要保 SLA

中文译文（Sam Mutu） 而从超大规模运营者这边看，挑战也许在于：所有这些解决的都是"从今天到未来"，但我们有一支舰队要维持——那才是最大的挑战。

（Raj） 你的舰队要跑多少年？ **（Sam）** 十年。

中文译文（Raj Kapoor） 这就回到我前面说的那点：我们会在这里待很久，这件事不会明天就解决。所以等到我们真的看到这类 CVE 从数据库里被消除的时候，你们的舰队大概已经在 100% 跑 Rust 了。到那时我们就可以退休了。

中文译文（Tim Lewis） 但正如你正确指出的，这是一个攻击面的问题。要归零会非常难。但如果我们能移除很多——正如 Raj 说的，一点一点移除——我们就会进入更好的状态。

中文译文（Sam Mutu） 那么问题来了：我们的项目是不是被构造成可以"一点一点来"？还是说唯一的方式是一次性全做？那永远不会奏效，我们就永远卡在这里。所以我们必须想办法做到增量，否则就卡住了。

> "the challenge maybe butter and can to from a hyperscaler side is this is all solving from today's to future >> but we have a fleet to sustain >> and that is the biggest challenge. >> Sure. >> How many years how many years for your fleet? >> Decade."
> "to the point that I made earlier was that we are going to be here >> uh for the long haul. It's not going to be solved tomorrow. >> Yep. >> And so by the time we actually see these type of CVS getting eliminated uh from from from the um database um uh your your fleets are going to be you know running 100% rust. Then we can then we can retire."
> "but I think team as you as you rightfully mentioned it's it's a question of attack surface. Yes, we it's going to be very hard to reach zero. But if we can remove a lot and like you mentioned Raj n like remove a little bit at a time we're going to get in a better state."
> "are our projects built in such a way that we can do a little bit at a time or are we are we you know the only way we can do it is the whole thing one time that will never work right so we've got to figure out a way some way to do that because if we don't we'll we'll be stuck here"

> **小结** 三重约束：①舰队寿命以"十年"计，必须边跑边改；②即使写 Rust，若底层规范本身不是为内存安全语言设计的，仍会残留 `unsafe` 段；③目标不是归零，而是持续压缩攻击面，因此"可增量推进"的工程结构是前提而非优化项。

七　OCP / OPF 能做什么

7.1　把"新代码用 Rust"写成最佳实践

中文译文（Raj Kapoor） OCP 能做的是给我们提供 AI 工具，让我们可以开始做实验。（笑）玩笑归玩笑，我觉得他们是相当认真的。我认为我们需要在 OPF 项目下投入一些认真的工作。也许这是一个很好的起点。我们在 OPF 里已经讨论过的一件事就是：新代码用 Rust 写。

除了这个，还有没有一些指导方针，比如"这些是必须满足的指标"？——新代码从 Rust 开始。我们当然可以明确规定什么是最佳实践。哪怕只是说：最佳实践就是，你必须以内存安全的方式来写，并且这些是我们见过的、被证明有效的推荐工具。给出一些建议，而不是把每个人都丢在那里自己去摸索 Rust——"那我从哪开始？Rust 是不是……"

（主持） 换句话说，我们应该宣传 Rust。

> "OCP what OCP can do is uh is provide us with AI tools so that we can start the experimentation. [laughter] >> Good way to say AI. >> but I think I think I think they are um uh uh you know jokes apart um they are pretty serious about that and uh I think we need to we need to u you know get some um get get some some some serious um effort under the OPF project as well. maybe this is a great avenue to get started and uh one of the things that we've already discussed in our OPF is uh hey like I said you know new code write it in rust um now from a >> in addition to that is there any guidelines or anything that we need to put hey these are the uh metrics that need to match >> new code start with rust

> >> I think new code start >> we can certainly specify what this this is best this is best practice right >> best practice even just to say that best practice is we've got You've got to write it in such a way that it can be that it's memory safe, right? And figure and and these are some recommended tooling that you can use that we've seen have been successful, right? As we give some recommendations, not just leaving it up to everyone's just finding about Rust. Now, where do I start? Does Rust"

> **小结**　OCP/OPF 的第一个抓手是"降低起步成本"：把最佳实践写成明确条目（新代码用内存安全语言 + 推荐工具链），而不是把每个团队留在"从哪开始"的空白里。这属于标准组织最擅长的动作——把共识固化为可引用的文本。

7.2　宣传语言的内建优势：不构建整镜像也能测

中文译文（Oleg Ilyasov）　它有一些别的语言没有的独特特性。比如测试——如果你谈代码测试，Rust 语言内建的这一特性是独特的：你可以只写一小段，来测试你想测的每一个函数、任何你想测的东西，甚至不用构建出最终的固件镜像，你可以提前就做。这是非常有用的特性。所以我们应该宣传它，也许在 OCP 的文档里某个地方我们应该说明：这是该走的路——最佳实践。

中文译文（Tim Lewis）　而且也许是以身作则。就像你说的，OCP 所做的——标准需要为那些内存安全的（实现）做好准备、支持它。

> "it has the features that are unique that no other languages have. Like for example the testing if you're talking about the testing of your code it's unique feature of the language rust inbuilt that you could just just write the pieces just to test every function you want anything you want >> without without even making the final image of the firmware you you you can do it up front which is very useful feature. So we should advertise it and and maybe in the documents of OCP somewhere we should say that this is way to go >> best best practice. Yeah. >> And maybe by example also you know it's about living like you mentioned like what OCP does the standard needs to be ready for those memory safe VMware support it."

> **小结**　一个具体的、可验证的卖点：Rust 的内建测试框架允许在未构建完整固件镜像的前提下对单个函数做单元测试——这对固件这种"镜像构建慢、上线风险高"的场景价值极大。标准文档若把这类具体收益写清楚，比抽象地推荐语言更有效。

八　AI 辅助转换：四类 agent 的流水线

中文译文（Raj Kapoor）　大约六七个月前，我在琢磨"怎么把 C 转成 Rust"，那是一段相当艰难的探索。我的结论是：AI 并不擅长直接把你带到你想去的地方，所以你必须部署一些 agent。

首先你需要分析你的代码——AI agent 需要先分析，所以你要有一个**分析 agent**，由它来分析你的代码。当然，如我所说，从代码里较小的一个片段开始。然后，一旦分析完成，你需要一个**架构 agent**：它把分析出来的东西转换成意图——架构意图。等这个意图被文档化之后，你用第三个 agent，也就是**编码 agent**，由它来处理那份文档。

……一旦完成，最后一步非常明显：编译。所以你得有一个**编译 agent**；如果它产生错误，就回到前面去。我说的就是，大致这样的事是应该做的。

> "when I was toying around with um you know how do I convert C to Rust right um that was that was quite a quest but um what I concluded was that AI is not great at getting you to the to the point where you want to be so you'll have to deploy some um some agents, right? So you first would need to uh analyze your code. The AI agent need to analyze. So you have an analyzer agent, right? That analyzes your code. >> And of course, like I said, start with a fun a smaller section of your code. >> Then once it has been analyzed, then you need an architect agent, right? So this architect agent is going to take whatever it analyzed it would convert it into the intent the architecture intent. Once that is documented then you use the third uh agent which is coding agent that would take that documentation. >> Are we going to change your job title [laughter] >> because it's no it's chief arch architect architect. Okay. So, so, so anyway, so let me so to to complete so you have the third agent which is coding agent and once that is done what is the final step very very uh obvious compiling right so now you have to have a compiling agent and if that generates an error you go back to you know but something to that effect is what I'm saying should be done"

> **小结**　单靠一个大模型直接"翻译 C 到 Rust"不可行；可行的是分阶段流水线：分析 agent（读懂代码）→ 架构 agent（提炼意图）→ 编码 agent（按意图生成）→ 编译 agent（验证并回环）。关键前提是"从小片段开始"，因为整库转换的上下文超出当前模型能力。

九　观众问答

中文译文（观众 Daniel）　也许这是个错误的前提，但我还是问一下，你也可以纠正这个前提。感觉推动这件事的主要动力是：AI 非常擅长发现这些不安全的内存问题，我们正涌来大量 CVE，造成了太多折腾，所以让我们转向 Rust 来预防这一类漏洞，对吧？

中文译文（Raj Kapoor）　我不认为 AI 在这方面是专家。根据我目前看到的情况，我可以肯定地告诉你。它能做一些，但它无法完成你 100% 的工作——那项"找漏洞"的工作。

中文译文（Giri Mudusuru）　这里要补充几点。我们需要有一些工具，让你在写的时候不会撞上安全问题；同时你还是得用 AI，不管它是 C 还是 Rust 还是你选的任何其他语言。你需要把漏洞找出来。也许该纠正这个前提：如你所说，这不是一个 AI 问题——漏洞在 AI 之前就存在。AI 起到的是乘数效应：它让发现的东西更多、更快，也让修复更快，这是好事。所以更准确地说，正如 Tim 前面提到的，我们要做的是消除一整类漏洞——而这恰好是固件领域最常见（如果不是最常见）的一类。而 AI 在这里，我把它看作助推这次转型的力量，因为如果没有更快的改动、编码和翻译，就做不到。除了"发现"那部分——那部分一直是最大的。

中文译文（Tim Lewis）　如果你看——你大概是在暗示 Mythos 之类的东西——是的，它能把你带到某个地方，但它不会把你带到终点。

中文译文（观众追问）　我本来想问的是：你认为这件事大部分会是"用 AI 做漏洞分诊"，然后我们转向 Rust；还是说干脆就靠 AI 去把那一类漏洞挑出来，效果一样？

> "Hello. Oh, look at that. Okay, cool. Uh so maybe a bad premise but I'll go ahead and ask this and feel free to correct the premise as well but it feels like um the main initiative that's driving this is hey AI is very good at finding these uh bad

> unsafe memory issues. We're having an influx of huge amounts of CVEEs and it's causing so much churn. Let's pivot to Rust to prevent this category of vulnerability. Right? >> I don't think AI is uh is expert in doing that. I can I can tell you for certain based on what I've seen so far. Um >> but a can it can do some it can do some but it cannot do 100% of your job right of that job to >> find vulnerabilities. Yeah. >> Well, yeah. I mean, >> yeah, I think couple of things to add there, right? We need to have some tools when you are writing, you're not running into uh security problems and still you have to use AI regardless of uh if it's available, right? C or whatever R or whatever other language you choose, right. Right. You need to figure out the vulnerabilities. Maybe to correct the premise like you mentioned it's not an AI problem like vulnerabilities have been existing before AI AI has a multiplier effect as to what is found and how fast etc >> and also how fast we fix them which is good so it's it's more about I think team mentioned earlier it's eliminating a class of vulnerability that's what we're trying to do and it happens to be one of the most if not the most common class in the firmware space >> and AI here I see as the helping aiding this transition because without that faster change or coding and translation, right? That's how I see AI. But other than the discovery part, which is yeah, always the biggest." "if you look at I mean you're probably hinting at Mythos or you know those those type of Yeah. Um it you know it it will get you somewhere but you know it won't get you all the way you know."

> **小结**　三点澄清：①漏洞在 AI 之前就存在，AI 只是"乘数"——既加快发现，也加快修复；②真正的目标是"消除一整类缺陷"，而内存安全恰好是固件中最常见的一类；③AI 的价值主要在加速改造与转换，而不是替代结构性治理。用 Tim 的话说：能带你走一段，但走不到终点。

附录　时间轴

时间	内容
00:07	开场：安全启动不等于固件内部安全
01:14	嘉宾自我介绍（Oracle / Microsoft / Insyde / Google / AMI / AMD）
02:39	议题一：为什么现在要关注内存安全
04:06	Raj：增量改良已失效
04:30	Tim："完美风暴"三因素
06:19	Giri：存量舰队与 SLA 压力
07:21	Bertrand：固件被忽视但权限极高
08:10	Oleg：越往栈底越难修
09:18	议题二：换语言重编译够不够
10:34	把发现前移到编译期
11:35	Tim：让编译器替管理者说话

时间	内容
13:26	议题三：指针与 unsafe 的可审计性
15:18	unsafe 占比可作为度量与告警
16:00	议题四：迁移路径，从新代码开始
18:35	DARPA TRACTOR 计划
19:44	模块化：优先内存管理相关部分
20:46	议题五：供应链与人才
21:44	Microsoft Patina 开源
22:42	议题六：存量舰队与规范约束
24:38	目标不是归零，而是压缩攻击面
25:35	议题七：OCP/OPF 的最佳实践抓手
27:31	议题八：四类 agent 的转换流水线
29:51	观众问答：AI 是否是驱动力
32:50	结束